Barendregt's λ-Cube

Lecture 14

Section 1

Part I: Where We Stand

The story so far

We have built a tower of increasingly expressive type systems:

System	**Key idea**	**Logic (C-H)**	**New axis**
STLC	$\lambda x{:}A.\, t$	propositional	—
System T	Nat, rec	quant.-free PA*	data + recursion
System F	$\Lambda\alpha.\, t$	2nd-order prop.	terms depend on types
System Fω	type operators	higher-order prop.	types depend on types

{*Via Dialectica, not direct C-H.}

Monads showed us how to *use* this machinery in practice. Today we step back and ask: **is there a unifying framework?**

The proliferation problem

For each system, we have had to:

- define a separate syntax (STLC has $\lambda x{:}A.\, t$; System F adds $\Lambda\alpha.\, t$ and $t\,[\![A]\!]$; Fω adds $\lambda\alpha{:}\kappa.\, T$ at the type level)
- state separate typing rules
- prove properties like strong normalization separately — we did it for STLC; for System F one would need a substantially different argument

The proliferation problem

For each system, we have had to:

- define a separate syntax (STLC has $\lambda x{:}A.\, t$; System F adds $\Lambda\alpha.\, t$ and $t\,[\![A]\!]$; Fω adds $\lambda\alpha{:}\kappa.\, T$ at the type level)
- state separate typing rules
- prove properties like strong normalization separately — we did it for STLC; for System F one would need a substantially different argument

Every time we add a new axis, the whole development must in principle be redone. Can we do better?

Four forms of dependency

Look at the systems we have built. Each one adds a new form of **dependency** between terms and types:

Dependency	**Notation**	**System**
terms depend on terms	$\lambda x{:}A.\, t$	STLC
terms depend on types	$\Lambda\alpha.\, t$	System F
types depend on types	$\lambda\alpha{:}*.\, T$	System Fω
types depend on terms	???	???

Four forms of dependency

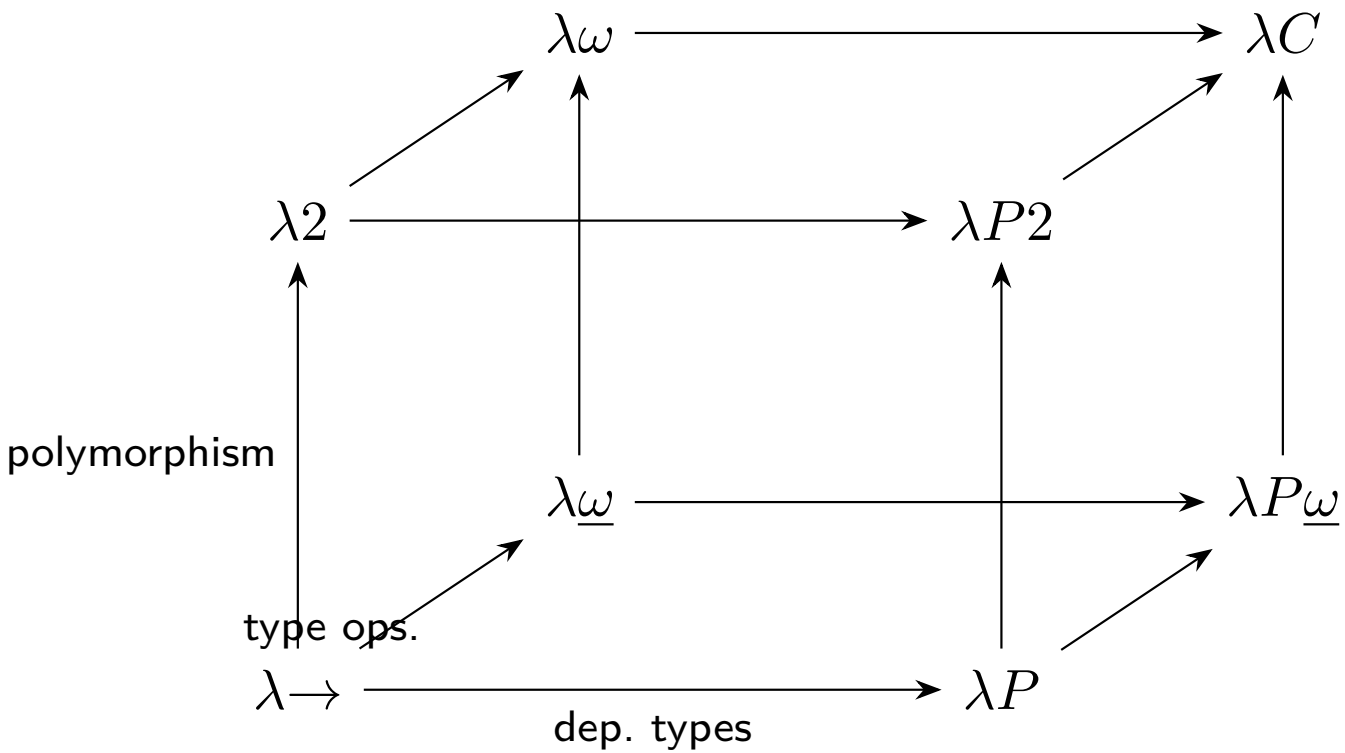
Look at the systems we have built. Each one adds a new form of **dependency** between terms and types:

Dependency	**Notation**	**System**
terms depend on terms	$\lambda x{:}A.\, t$	STLC
terms depend on types	$\Lambda\alpha.\, t$	System F
types depend on types	$\lambda\alpha{:}*.\, T$	System Fω
types depend on terms	???	???

There are exactly four combinations. We have seen three. A natural question: can we organize all four into a single framework?

The λ-cube: a first look

Barendregt's insight: arrange these four dependencies as axes of a cube.



Each vertex is a type system. Each edge adds one form of dependency. The bottom-left corner is STLC; the top-right corner has everything.

Abstraction that clarifies

All four forms of dependency are instances of the same idea. The cube uses just three expression forms:

- $\Pi x{:}A.\, B$ — the **product**: given something x classified by A, the result is classified by B (which may mention x)
- $\lambda x{:}A.\, b$ — the **abstraction**: an expression that *inhabits* a product; it binds x in the body b
- $f\ a$ — the **application**: feeds a to f, producing a result classified by $B\{x := a\}$

The pattern generalizes $A \to B$ and $\lambda x{:}A.\, b$ from STLC: Π describes the interface, λ provides the implementation.

What makes this different from STLC

In STLC, A and B in $A \to B$ are always types, and $\lambda x.\, b$ is always a term. In the cube, A and B can live at **any level**:

A **is...**	**so** x **is...**	**Example**
a type (e.g. Nat)	a term	$\lambda x{:}\mathsf{Nat}.\, x + 1$
a kind (e.g. $*$)	a type	$\lambda \alpha{:}*\,.\, \alpha \to \alpha$
a kind (e.g. $* \to *$)	a type constructor	$\lambda F{:}(* \to *).\, F\ \mathsf{Nat}$

The same Π and λ handle all cases. What is new is that A and B may live at different levels, and B may depend on x.

The simplification

In previous lectures we accumulated separate notation for each system: λ and Λ for two kinds of abstraction, $\forall$ and $\to$ and kind arrows for three kinds of product, $t\,[\![A]\!]$ for type application vs. ordinary application.

In the cube, **all of these collapse**. $\Lambda\alpha.\,t$ is just $\lambda\alpha{:}*\,.\,t$. $\forall\alpha.\,B$ is just $\Pi\alpha{:}*\,.\,B$. Type application $t\,[\![A]\!]$ is just $t\ A$. The arrow $A \to B$ is just $\Pi x{:}A.\,B$ with x not appearing in B.

The framework is not just more general — it is *simpler*. The abstraction reveals that the systems we built are variations on a single theme.

Let us now make this precise.

Section 2

Part II: The Unified Language

Two sorts

The cube's language has exactly two **sorts**:

Sort	**Symbol**	**Classifies**	
star	$*$	types	$(\text{Nat} : *, \quad \text{Nat} \to \text{Nat} : *)$
box	$\Box$	kinds	$(* : \Box, \quad * \to * : \Box)$

The single axiom is $* : \Box$ — "the collection of all types is a kind.''

Terms have types (sort $*$), and types have kinds (sort $\Box$). The cube makes this hierarchy precise.

Expressions

For each sort $s \in \{*, \Box\}$ there is a countably infinite set of variables $\mathcal{V}_s$: term variables $x, y, z \in \mathcal{V}_*$ and type variables $\alpha, \beta, \gamma \in \mathcal{V}_\Box$, with $\mathcal{V}_* \cap \mathcal{V}_\Box = \emptyset$.

The set $\mathcal{E}$ of **expressions** is:

$$\mathcal{E} \;=\; \mathcal{V} \;\mid\; \mathcal{S} \;\mid\; \mathcal{E}\,\mathcal{E} \;\mid\; \lambda \mathcal{V}{:}\mathcal{E}.\,\mathcal{E} \;\mid\; \Pi \mathcal{V}{:}\mathcal{E}.\,\mathcal{E}$$

Five forms, nothing more. No separate Λ, no separate $\forall$, no kind-level arrow. The sorts determine what is a `term,'' what is atype,''` and what is a "kind.''

What happened to Λ and $\forall$?

The old notations become special cases of λ and Π:

Old notation	$=$	**Cube notation**
$\forall \alpha . B$		$\Pi \alpha : * . B$
$\Lambda \alpha . t$		$\lambda \alpha : * . t$
$t \, [\![A]\!]$ (type application)		$t \; A$ (ordinary application)
$\lambda \alpha : \kappa . T$ (type operator)		$\lambda \alpha : \kappa . T$ (unchanged)

What used to require five or six different syntactic forms is now just two: λ and Π.

The arrow abbreviation

Whenever $x \notin \mathsf{FV}(B)$, we write:

$$\Pi x{:}A.\, B \quad \equiv \quad A \to B$$

This is the **same** convention at every level:

- Nat $\to$ Nat abbreviates $\Pi x{:}\mathsf{Nat}.\, \mathsf{Nat}$ (function type, sort $*$)
- $* \to *$ abbreviates $\Pi \alpha{:} * .\, *$ (kind, sort $\Box$)

There is nothing new about $\to$ at the kind level — it is the same non-dependent Π, just applied to expressions of a different sort.

β-reduction

There is a single notion of reduction:

$$(\lambda x{:}A.\, M)\ N \ \rightarrow_\beta \ M\{x := N\}$$

This covers all the old reduction rules at once:

- $(\lambda x{:}\mathsf{Nat}.\, x + 1)\ 3 \ \rightarrow_\beta \ 3 + 1$ (term applied to term)
- $(\lambda \alpha{:}*\,.\, \lambda x{:}\alpha.\, x)\ \mathsf{Nat} \ \rightarrow_\beta \ \lambda x{:}\mathsf{Nat}.\, x$ (old: $\Lambda\alpha$ applied to type)
- $(\lambda \alpha{:}*\,.\, \alpha \rightarrow \alpha)\ \mathsf{Nat} \ \rightarrow_\beta \ \mathsf{Nat} \rightarrow \mathsf{Nat}$ (type operator applied)

One rule. Three old rules as special cases.

Section 3

Part III: The Four Rules

Rules as toggles

The cube's product rule asks: when can we form $\Pi x{:}A.\, B$? The answer depends on the **sorts** of A and B:

s_1	s_2	**Meaning**	**Read $\Pi x{:}A.\, B$ as...**
$*$	$*$	terms depending on terms	function type
$\Box$	$*$	terms depending on types	polymorphic type
$\Box$	$\Box$	types depending on types	type-operator kind
$*$	$\Box$	types depending on terms	dependent kind

The rule $(*, *)$ is always present. The other three are optional toggles.

The product rule

For any set of rules $\mathcal{R} \subseteq \{*, \Box\} \times \{*, \Box\}$:

$$\frac{\Gamma \vdash A : s_1 \qquad \Gamma, x{:}A \vdash B : s_2}{\Gamma \vdash (\Pi x{:}A.\, B) : s_2} \quad \text{if } (s_1, s_2) \in \mathcal{R}$$

The sort of the product is s_2 — the sort of the *body*. If the body is a type ($s_2 = *$), then the product is a type; if the body is a kind ($s_2 = \Box$), then the product is a kind.

The inference rules (structural)

These rules are shared by all eight systems:

(axiom) $[] \vdash * : \Box$

(start) $$\frac{\Gamma \vdash A : s}{\Gamma, x{:}A \vdash x : A} \quad \text{if } x \in \mathcal{V}_s,\ x \notin \mathsf{dom}(\Gamma)$$

(weakening) $$\frac{\Gamma \vdash A : B \quad \Gamma \vdash C : s}{\Gamma, x{:}C \vdash A : B} \quad \text{if } x \in \mathcal{V}_s,\ x \notin \mathsf{dom}(\Gamma)$$

The inference rules (Π, λ, application, conversion)

(product) $$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x{:}A \vdash B : s_2}{\Gamma \vdash (\Pi x{:}A.\, B) : s_2} \quad \text{if } (s_1, s_2) \in \mathcal{R}$$

(abstraction) $$\frac{\Gamma, x{:}A \vdash b : B \quad \Gamma \vdash (\Pi x{:}A.\, B) : s}{\Gamma \vdash \lambda x{:}A.\, b : \Pi x{:}A.\, B}$$

(application) $$\frac{\Gamma \vdash F : (\Pi x{:}A.\, B) \quad \Gamma \vdash a : A}{\Gamma \vdash F\ a : B\{x := a\}}$$

(conversion) $$\frac{\Gamma \vdash A : B \quad \Gamma \vdash B' : s}{\Gamma \vdash A : B'} \quad \text{if } B =_\beta B'$$

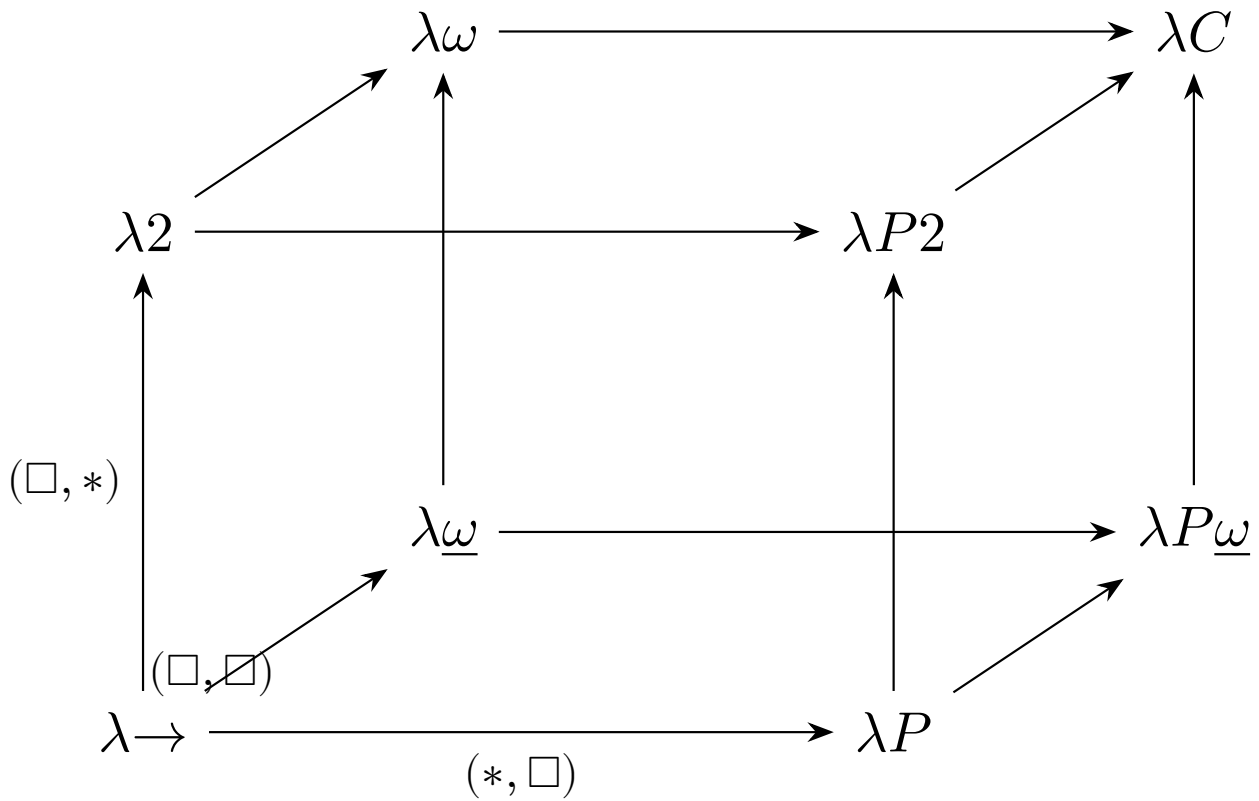
The **only** rule that mentions $\mathcal{R}$ is the product rule. Everything else is fixed.

The eight systems

System	$(*, *)$	$(\Box, *)$	$(\Box, \Box)$	$(*, \Box)$
$\lambda\!\!\rightarrow$ (STLC)	✓			
$\lambda 2$ (System F)	✓	✓		
$\lambda\underline{\omega}$	✓		✓	
$\lambda\omega$ (System Fω)	✓	✓	✓	
λP	✓			✓
$\lambda P2$	✓	✓		✓
$\lambda P\underline{\omega}$	✓		✓	✓
λC (Calc. of Constr.)	✓	✓	✓	✓

Three binary toggles, $2^3 = 8$ systems.

The cube, geometrically



Right = dependent types $(*, \Box)$. **Up** = polymorphism $(\Box, *)$. **Forward** = type operators $(\Box, \Box)$.

Section 4

Part IV: Recovering STLC

$\lambda\!\!\rightarrow$ is the simply typed λ-calculus

Set $\mathcal{R} = \{(*, *)\}$. The only permitted product is:

$$\frac{\Gamma \vdash A : * \qquad \Gamma, x{:}A \vdash B : *}{\Gamma \vdash (\Pi x{:}A.\, B) : *}$$

Both A and B live in sort $*$, so both are types.

Claim: in $\lambda\!\!\rightarrow$, x can *never* appear free in B.

$\lambda\!\rightarrow$ is the simply typed λ-calculus

Set $\mathcal{R} = \{(*, *)\}$. The only permitted product is:

$$\frac{\Gamma \vdash A : * \qquad \Gamma, x{:}A \vdash B : *}{\Gamma \vdash (\Pi x{:}A.\, B) : *}$$

Both A and B live in sort $*$, so both are types.

Claim: in $\lambda\!\rightarrow$, x can *never* appear free in B.

Why? With only the rule $(*, *)$, there is no way for a term variable to appear in a type expression. Types cannot mention terms. So $\Pi x{:}A.\, B$ is always just $A \rightarrow B$.

Derivation: the identity in $\lambda\!\rightarrow$

We derive $\alpha : * \vdash \lambda x{:}\alpha.\, x : \alpha \to \alpha$:

1. $[] \vdash * : \Box$ (axiom)
2. $\alpha : * \vdash \alpha : *$ (start, using (1))
3. $\alpha : *,\ x : \alpha \vdash x : \alpha$ (start, using (2))
4. $\alpha : * \vdash (\Pi x{:}\alpha.\, \alpha) : *$ (product $(*, *)$, using (2))
5. $\alpha : * \vdash \lambda x{:}\alpha.\, x : \alpha \to \alpha$ (abstraction, using (3) and (4))

The type $\Pi x{:}\alpha.\, \alpha$ is $\alpha \to \alpha$ since $x \notin \mathrm{FV}(\alpha)$.

What $\lambda\!\rightarrow$ cannot do

We **cannot** close the identity off with $\Pi\alpha{:}*\,.\,(\alpha \to \alpha)$:

$$\frac{\Gamma \vdash * : \Box \qquad \Gamma, \alpha{:}* \vdash \alpha \to \alpha : *}{\Gamma \vdash (\Pi\alpha{:}*\,.\,\alpha \to \alpha) : *} \quad \text{requires } (\Box, *) \notin \mathcal{R}_{\lambda\!\rightarrow}$$

The type $\forall\alpha.\,\alpha \to \alpha$ is **not expressible** in $\lambda\!\rightarrow$. The identity must be given at a fixed type, just as in STLC.

Section 5

Part V: Recovering System F

$\lambda 2$ is System F

Set $\mathcal{R} = \{(*, *),\ (\Box, *)\}$. The new rule allows polymorphism:

$$\frac{\Gamma \vdash A : \Box \qquad \Gamma, \alpha{:}A \vdash B : *}{\Gamma \vdash (\Pi \alpha{:}A.\, B) : *} \quad (\Box, *)$$

Since $A : \Box$, the variable α ranges over types. Since $B : *$, the product is itself a type. This is **quantification over types**.

The polymorphic identity in $\lambda 2$

We extend the earlier derivation with the new rule:

1. $[] \vdash * : \Box$ (axiom)
2. $\alpha : * \vdash \alpha : *$ (start)
3. $\alpha : * \vdash (\Pi x{:}\alpha.\, \alpha) : *$ (product, $(*, *)$)
4. $\alpha : * \vdash \lambda x{:}\alpha.\, x : \alpha \to \alpha$ (abstraction)

Steps 1–4 are the same as in $\lambda{\to}$. Now the crucial new step:

5. $[] \vdash (\Pi \alpha{:} * .\, \alpha \to \alpha) : *$ **(product, $(\Box, *)$, using (1) and (3))**
6. $[] \vdash \lambda \alpha{:} * .\, \lambda x{:}\alpha.\, x : \Pi \alpha{:} * .\, \alpha \to \alpha$ (abstraction)

Reading the derivation

The type $\Pi\alpha{:}*\,.\,\alpha \to \alpha$ is exactly $\forall\alpha.\,\alpha \to \alpha$.

The term $\lambda\alpha{:}*\,.\,\lambda x{:}\alpha.\,x$ is what we used to write $\Lambda\alpha.\,\lambda x{:}\alpha.\,x$.

Type application is now ordinary application:

$$(\lambda\alpha{:}*\,.\,\lambda x{:}\alpha.\,x)\ \mathsf{Nat}$$

The application rule gives result type $(\alpha \to \alpha)\{\alpha := \mathsf{Nat}\} = \mathsf{Nat} \to \mathsf{Nat}$.
No separate "type application'' rule is needed.

Every System F derivation translates mechanically into a $\lambda 2$ derivation.

Church-encoded naturals in $\lambda 2$

Recall: $\mathsf{Nat}_{\mathsf{Ch}} = \forall\alpha.\,(\alpha \to \alpha) \to \alpha \to \alpha.$

In the cube: $\mathsf{Nat}_{\mathsf{Ch}} = \Pi\alpha{:}*\,.\,(\alpha \to \alpha) \to \alpha \to \alpha.$

The Church numeral $\bar{2} = \lambda\alpha{:}*\,.\,\lambda f{:}(\alpha \to \alpha).\,\lambda x{:}\alpha.\,f\ (f\ x).$

Type-checking in context $\alpha : *,\ f : \alpha \to \alpha,\ x : \alpha$:

- $f\ x : \alpha$ and $f\ (f\ x) : \alpha$ by repeated application
- Abstracting x, f, α in order gives type $\mathsf{Nat}_{\mathsf{Ch}}$ ✓

What $\lambda 2$ still cannot do

With only $(*, *)$ and $(\Box, *)$, we cannot form **type-level functions**. The kind $* \to *$ requires:

$$\frac{\Gamma \vdash * : \Box \qquad \Gamma, \alpha : * \vdash * : \Box}{\Gamma \vdash (\Pi \alpha{:}\, * \,.\, *) : \Box} \quad \text{requires } (\Box, \Box) \notin \mathcal{R}_{\lambda 2}$$

So type operators like List $: * \to *$ cannot be defined as λ-expressions at the type level. We need the next rule.

Section 6

Part VI: Recovering System Fω

$\lambda\omega$ is System Fω

Set $\mathcal{R} = \{(*, *),\ (\Box, *),\ (\Box, \Box)\}$. The new rule:

$$\frac{\Gamma \vdash A : \Box \qquad \Gamma, \alpha{:}A \vdash B : \Box}{\Gamma \vdash (\Pi \alpha{:}A.\, B) : \Box} \quad (\Box, \Box)$$

Now both domain and codomain are kinds. This lets us form kind-level arrows $* \to *$, $* \to * \to *$, $(* \to *) \to *$, etc.

Derivation: the kind $* \longrightarrow *$

In the cube, $* \longrightarrow *$ is $\Pi\alpha{:}* \,.\, *$. Derivation:

1. $[] \vdash * : \Box$ (axiom)
2. $\alpha : * \vdash * : \Box$ (weakening, using (1) and (1))
3. $[] \vdash (\Pi\alpha{:}* \,.\, *) : \Box$ **(product, $(\Box, \Box)$, using (1) and (2))**

So $* \longrightarrow * : \Box$ — it is a valid kind. This was **impossible** in $\lambda 2$.

A type operator in $\lambda\omega$

Define $D = \lambda\alpha{:}* \,.\, \alpha \to \alpha$. We derive $D : * \to *$:

1. $\alpha : * \vdash \alpha : *$ (start)
2. $\alpha : * \vdash (\alpha \to \alpha) : *$ (product $(*, *)$)
3. $[] \vdash (* \to *) : \Box$ (derived on previous slide)
4. $[] \vdash \lambda\alpha{:}* \,.\, (\alpha \to \alpha) : * \to *$ (abstraction, using (2) and (3))

Applying it: D Nat $=_\beta$ Nat $\to$ Nat. The **conversion rule** lets us use D Nat and Nat $\to$ Nat interchangeably.

Using the conversion rule

Given $D = \lambda\alpha{:}* \,.\, \alpha \to \alpha$:

$$\alpha : * \;\vdash\; \lambda x{:}(D\ \alpha).\, x \;:\; D\ \alpha \to D\ \alpha$$

Since $D\ \alpha =_\beta \alpha \to \alpha$, the conversion rule also gives:

$$\alpha : * \;\vdash\; \lambda x{:}(D\ \alpha).\, x \;:\; (\alpha \to \alpha) \to (\alpha \to \alpha)$$

The type checker must β-reduce $D\ \alpha$ to verify this. This is a feature of every cube system with $(\Box, \Box)$: **type-level computation** forces the type checker to do β-reduction.

Summary: the left face of the cube

Cube vertex	**Rules $\mathcal{R}$**	**Old system**
$\lambda\!\!\rightarrow$	$\{(*,*)\}$	STLC
$\lambda 2$	$\{(*,*),\ (\Box,*)\}$	System F
$\lambda\omega$	$\{(*,*),\ (\Box,*),\ (\Box,\Box)\}$	System Fω

The cube did not change these systems. It **re-derived** them as special cases of one uniform definition. Old syntax, old rules, and old derivations all translate directly.

The payoff: one proof for all

Consider **subject reduction**: if $\Gamma \vdash t : A$ and $t \to_\beta t'$, then $\Gamma \vdash t' : A$.

In the old approach, we prove this separately for each system, adapting to different syntax and rules each time.

In the cube, we prove it **once**: by induction on the derivation of $\Gamma \vdash t : A$. At no point does the proof inspect which pairs are in $\mathcal{R}$. The argument is parametric.

The same holds for Church–Rosser, the substitution lemma, the generation lemma, correctness of types, and — with more work — strong normalization.

Section 7

Part VII: The Missing Axis

What about the right face?

We have explored the left face ($\lambda{\to}$, $\lambda 2$, $\lambda\omega$). The right face adds $(*, \Box)$ — types depending on terms. None of our systems have this.

Rule	**Dependency**	**Status**
$(*, *)$	terms on terms	✓ always present
$(\Box, *)$	terms on types	✓ present in $\lambda 2$, $\lambda\omega$
$(\Box, \Box)$	types on types	✓ present in $\lambda\underline{\omega}$, $\lambda\omega$
$(*, \Box)$	types on terms	**✗ not yet seen**

What would it mean for a type to depend on a term?

A preview: types that mention values

- Vec $\alpha\ n$ — vectors of length n, where n : Nat is a **term**
- Matrix $\mathbb{R}\ m\ n$ — $m \times n$ matrices
- Fin n — the type with exactly n elements

In System F, all lists have the same type regardless of length. We cannot express: "this function returns a vector of the same length as its input.''

The rule $(*, \Box)$ makes this possible. This is the system λP, the subject of the next lecture.

Section 8

Summary

What we covered

1. **The motivation**: the λ-cube gives a uniform framework where one proof covers all systems.
2. **The unified syntax**: two sorts $\{*, \Box\}$, five expression forms, one β-rule. The arrow $\to$ is always just non-dependent Π.
3. **The four rules**: each pair (s_1, s_2) enables a form of dependency.
4. **Recovering STLC**: $\lambda{\to}$ with $\{(*, *)\}$.
5. **Recovering System F**: $\lambda 2$ with $\{(*, *), (\Box, *)\}$.
6. **Recovering System Fω**: $\lambda\omega$ with $\{(*, *), (\Box, *), (\Box, \Box)\}$.

Next time

Next lecture: the system λP — dependent types.

We will explore the rule $(*, \Box)$ in detail: types depending on terms, the first-order universal quantifier $\Pi x{:}A.\, B(x)$, predicates and relations as types, and the Curry–Howard correspondence for predicate logic.

This is the fourth and final axis of the cube — the one that opens the door to the Calculus of Constructions.